Appendix I — Reading the Assembly

Ghidra's C is a guess at the calling convention, the width of each value, and which register is which argument. The bytes are the program. When the C and a live dump disagree, the dump wins. When the C and the disassembly disagree, the disassembly wins.

Date: 16 October 2026

This is the minimum needed to check a listing while stopped in SoftICE or WDW. It is not a general 8086 manual.

I.1 An address is two numbers

A far address is `segment:offset`. In real mode the physical address is `segment × 16 + offset`. In the Win98 protected-mode sessions the segment number is a selector. The CPU, not you, turns it into a base. Two different selectors can point at the same NE segment on two boots. The offset inside the segment does not change.

Ghidra's name `FUN_1160_03de` means offset `03DE` in the Ghidra segment whose base is `1160`. That base is not a live selector. NE segment `N` is usually Ghidra base `0x1000 + (N − 1) × 8`, with gaps where a segment is not code. Segment 1 is `1000`. Segment 2 is `1008`. Segment `0x2D` (45) is `1160`. Segment `0x2F` (47) is `1170`. Appendix D is the whole table.

A symbol map line `1160:03DE FUN_1160_03de` is this same pair. SoftICE and WDW do not load that file. You use it on paper.

I.2 Bytes are little-endian

The dump shows bytes in memory order. A word is stored low byte first.

Bytes in the dump	As a word	As text
`04 00`	`0004`	length 4
`4A 42`	`424A`	the characters `J` then `B` if you read bytes, not the word
`03 4A 42 4F`		`03`, then `JBO`
`14 00`	`0014`	a record type byte `0x14`, then a zero
`FC B5 BD B0`		the four bytes from the 15 October buffer

Read a word off a dump by swapping the two bytes. Read a signature by not swapping. `4F 46` at the next address is the characters `OF`. It is not part of the dword you already read at the previous address. The 15 October wider dump (`9070` for `0x30` bytes) showed `4F 46` at `9092` and `FC B5 BD B0` at `9086`. Those are different objects that happen to sit near each other.

I.3 The frame after ENTER

A far call pushes the return address as CS then IP, then the callee runs. ToolBook's compiled functions almost all start with `ENTER nn,00`. That instruction pushes BP, sets BP to SP, and subtracts `nn` from SP for locals.

After `ENTER` has executed, for a far callee:

Location	What it is
`[BP+0]`	Saved BP.
`[BP+2]`	Return IP.
`[BP+4]`	Return CS.
`[BP+6]`	First argument.
`[BP+8]`	Second argument.

Location	What it is
`[BP−2]` downward	Locals. Negative offsets are locals. Positive offsets past 4 are arguments.

A near call does not push CS. The first argument is then at `[BP+4]`. If you are not sure which you entered, look at the call instruction in the caller. `CALL 36F7:03DE` is far (it has a segment). `CALL 0CB1` with no segment is near. The Calls pane in WDW labels them. The code bytes do too: a far direct call is `9A`, a near direct call is `E8`.

Dump the frame only after `ENTER`. Before that instruction, BP still belongs to the caller, and `[BP+6]` is the caller's argument or a local. That was the wrong dump at `36E7:201B` on 15 October. The call had not been entered.

Two words that form a far pointer are pushed segment last, offset first, so that after the call the offset sits at the lower address. In the 4-byte read, the length was the last thing pushed (`PUSH 04`), so it is the first argument, at `[BP+6]`. The buffer pointer is the next argument: offset at `[BP+8]`, segment at `[BP+0A]`. Dumping `DW SS:BP+8 L 2` prints offset then segment, which is the order you need for `DB segment:offset`.

Ghidra often prints this as `param_1`, `param_2`, and a `local_4` that is actually the pointer. The names are positions. They are not types. `undefined2` means "two bytes, untyped." `undefined4` is often a far pointer the decompiler did not split. `unaff_CS` and `unaff_SS` mean "this register was live and the decompiler does not know who set it." Treat those as unknown, not as arguments you can pass.

I.4 Instructions that showed up

Enough to read a basic block. Sizes: a byte is 8 bits, a word is 16, a dword is 32.

Instruction	What it does	How it looks wrong in C
`MOV r, r/m`	Copy.	An assignment. Fine.
`PUSH imm` / `PUSH r`	Put a word on the stack.	A function argument. The order is right to left, so the last `PUSH` is the first argument.
`POP r`	Take a word off.	Often deleted from the C and folded into a return value.
`CALL far`	Push CS:IP, go.	A function call. The decompiler may invent the callee's signature.
`ENTER nn,00`	Make the frame in I.3.	The opening `{` of the function. The C will not show it.
`LEAVE`	Undo `ENTER`.	The closing of the function.
`RET` / `RETF`	Near or far return. `RETF n` also pops `n` bytes of arguments.	`return`.
`CMP a, b`	Set flags from `a − b`. Does not store the result.	An `if`. Look at the jump, not at the `CMP` alone.
`JZ` / `JNZ`	Jump if the compare was equal / not equal. Also written `JE` / `JNE`.	Which branch is the `if` body. Ghidra sometimes swaps them and then negates the test.
`ADD` / `SUB`	Arithmetic that is stored.	`+` / `−`.
`IMUL`	Signed multiply. `IMUL AX, imm16` multiplies AX by that immediate.	A scale. `IMUL AX, 0x14` is "times 20," which is a stride, not a coincidence.
`LEA`	Address, not a load.	`&` of something.
`INT 21` with AH=`3F`	DOS read. BX is the handle, CX is the length, DS:DX is the buffer.	A call the decompiler may name `DOS3CALL` or leave as a thunk.

`P` / `step` executes a `CALL` and stops on the next instruction in the caller. `T` / `trace` stops on the first instruction of the callee. Use `T` only when you mean to read that callee. A `CALL` into `KERNEL` traced with `T` is how the session GPFed.

I.5 One block, read as bytes

This is the call that asked for 4 bytes. Selectors are that boot.

```
36E7:2019  6A 04           PUSH 04
36E7:201B  9A DE 03 F7 36  CALL 36F7:03DE
```

`6A 04` pushes the sign-extended byte 4. `9A` is a far direct call. The next four bytes are offset then segment, little-endian: `DE 03` is offset `03DE`, `F7 36` is segment `36F7`. Ghidra shows the same call as `FUN_1160_03de(...)` and invents the

argument list. The length is the only thing this instruction pushes. Anything else the C lists was pushed earlier.

Inside the callee, after the buffer path returns, the branch that skipped the DOS read was:

```
048F   CALL FUN_1258_00a8
049A   JZ 0516
04F3   ; raw DOS read, not taken
```

`JZ` means “the zero flag is set, so skip ahead.” On that hit the flag was set, so `04F3` never ran. A C rendering that always shows both the buffered call and the DOS call as unconditional is wrong about control flow. The jump is the control flow.

I.6 A stride is a multiply, not a struct

The loop that started this search, at `35C7:02C2`, does this:

```
MOV  AX, [BX+SI+0E]
IMUL AX, 14
ADD  BX, AX
CMP  BYTE PTR [BX], 14
JNZ  <back>
```

Read it in words. Take the word at `[BX+SI+0x0E]`. Multiply by `0x14` (20 decimal). Add that to BX. Compare the byte at BX with `0x14`. If it is not `0x14`, do it again.

So each step moves 20 times some count, and the byte that must be `0x14` is the first byte of the thing landed on. That is a record stride of 20 bytes and a type byte of `0x14`. It does not require the three bytes `14 00 00` to sit in the file as a magic string the compiler compares in one instruction. A search for the immediate dword `14 00 00 00` can miss a compare of one byte against `0x14`.

Ghidra will often turn this into a `while` with casts through `undefined` and a multiply you have to find. The five instructions are the clearer statement. If the C does not multiply by 20 and then test a byte, the C is not this loop.

I.7 How the C lies, with the forms in these listings

Check these before you believe a body.

- The signature is invented. `__stdcall16far` is a good default for this compiler. The number and width of `param_N` is not. Count the `PUSH` instructions in the caller, and count the bytes a `RETF n` pops. Those two counts are the argument list.
- A far pointer is split. `param_1` as `undefined4`, then `(int)param_1` and `(undefined2)((ulong)param_1 >> 0x10)`, is offset and segment. In the debugger that is one `segment:offset`, not two integers you add.
- Locals are stack slots. `local_4`, `local_6`, `local_8` are `[BP-4]`, `[BP-6]`, `[BP-8]`. If the C uses `local_8` as a pointer and also as a flag, one of those uses is a register reuse the decompiler failed to split.
- **`unaff_CS` passed as an argument** means the decompiler lost a segment override or a live CS. Do not go looking for a source variable of that name. There is not one.
- **A thunk that shows `??`** is an ordinal import whose module was not bound in that Ghidra project. The instruction is still a far call. The export tables in Appendix A are where the ordinal gets a name. The C will not grow the name on its own.
- A switch on a byte (`if (local_4[0] == 0x38)` and a ladder of constants) is a type byte or a message. The constants are data. Write them down in hex before you accept the decompiler’s decimal.
- Dead code after a jump. Ghidra sometimes prints a block and then a comment that it is unreachable (`Removing unreachable block`). Do not trace that block live. It is not on the path.

- Absence. A function missing from an HTML or C export may be bytes the loader never turned into a function. That is an undisassembled run, not a deleted function. Force-disassemble recovers `ENTER` (`C8`) sites and a few prologues. It does not name them, and it does not prove they run on book open.

I.8 Check one claim, then stop

Pick one sentence of C. Find the offset Ghidra printed for it. In the debugger, `U` that offset (SoftICE) or set CS:IP's pane on it (WDW). You are looking for one of three outcomes.

1. The bytes match the sentence. Keep the sentence. Write the offset in the margin, not a paraphrase.
2. The bytes are a jump the C treated as certain. Keep the jump. Throw out the sentence.
3. The address is not code. You are in data, or in a segment that was not loaded. Do not name it.

The 4-byte buffer `FC B5 BD B0` is outcome 1 for the sentence "the wrapper reads the length at `[BP+6]` into the buffer at `[BP+8]`." It is outcome 2 for any sentence that says those four bytes are a record header. They are not `14 00`. They are not `JBO`. The wrapper is confirmed. The reader is still the function that, on a later hit, fills a buffer whose first byte is `0x14` and whose stride is `0x14`.

Do not promote a `FUN_` name to a purpose because the C compiled. Confirm the bytes, or leave the name as a location.